PROJECT REPORT

MOVIE RECOMMENDATION AND USER BEHAVIOR ANALYTICS

Using PySpark, Spark MLlib ALS, SQLite and Python



Department of Computer Science and Engineering

National Institute of Technology Delhi

(An autonomous Institute under the aegis of the Ministry of Education, Government of India)

Submitted by

Arun Kumar Singh | Roll No.: 231210025

Anshika Srivastav | Roll No.: 231210021

Aman Derwal | Roll No.: 231210012

Submitted to

Dr. Priten

Abstract

This project develops a movie recommendation and user behavior analytics workflow using a MovieLens-style dataset stored in SQLite. The data is loaded into Apache Spark DataFrames using PySpark, explored through descriptive analytics, and used to train a collaborative-filtering recommendation model with the Alternating Least Squares (ALS) algorithm available in Spark MLlib. The workflow examines movie popularity, average ratings, user activity, rating distributions, and rating-matrix sparsity. The trained model predicts ratings for held-out observations and is evaluated using Root Mean Squared Error (RMSE). It also generates top-N movie recommendations for users. The database used during development contains 8,570 movie records, 100,023 rating records, 8,570 link records, and 2,488 tag records. The project demonstrates an end-to-end Spark analytics and recommendation workflow on a local machine; it is a learning-scale implementation rather than a production-scale distributed deployment.

1. Introduction

Online movie platforms offer large catalogs, making it difficult for users to identify movies that match their interests. Recommendation systems address this problem by using historical interactions such as ratings to estimate which items a user may prefer. User behavior analytics provides additional insight into how users interact with the catalog.

This project combines exploratory analysis with collaborative filtering. SQLite is used as the local source database, PySpark provides DataFrame-based data processing, and Spark MLlib ALS learns latent user and movie representations from explicit ratings.

2. Problem Statement

The objective is to analyze movie-rating activity and build a model that can estimate users' ratings for movies they have not rated in the training data. The project also aims to summarize movie and user activity and present ranked movie suggestions for a selected user.

3. Objectives

- Load movie, rating, link, and tag records from a SQLite database.
- Create Spark DataFrames and inspect the data structure and record counts.
- Identify popular movies, highly rated movies, and active users.
- Visualize the distribution of ratings and selected activity measures.
- Measure the sparsity of the user–movie rating matrix.
- Train a collaborative-filtering model using Spark MLlib ALS.
- Evaluate predicted ratings using RMSE on a held-out test set.
- Generate and display top-N recommendations for a user.

4. Dataset Description

The project uses a MovieLens-style SQLite database named movielens-small.db. The database tables and record counts observed during project setup are shown below.

Table	Purpose	Records
movies	Movie identifiers, titles, years and genres	8,570
ratings	User ratings and timestamps for movies	100,023
links	External movie identifiers	8,570
tags	User-generated movie tags and timestamps	2,488

The ratings table provides the main interaction data for model training. The movies table supplies titles and genres for interpreting the recommendations. The links and tags tables are loaded and available for supplementary analysis.

Note: the dataset is relatively small and is used to demonstrate Spark APIs and MLlib. The project should not be described as a large production-scale cluster deployment.

5. Tools and Technologies

Technology	Role in project
Python	Main programming language.
Apache Spark / PySpark	Distributed-data processing API and DataFrame operations; run locally in this project.
Spark MLlib ALS	Collaborative-filtering algorithm for rating prediction and recommendations.
SQLite	Local relational database containing the source tables.
Pandas	Converting selected aggregated results for plotting.
Matplotlib	Visualizing exploratory analysis results.
Jupyter Notebook / VS Code	Interactive development and execution environment.

6. System Methodology

6.1 Database connection and data loading

The notebook connects to movielens-small.db with Python's sqlite3 module. It retrieves the records from movies, ratings, links, and tags, defines Spark schemas, and creates corresponding Spark DataFrames. Explicit schemas provide consistent column names and data types.

6.2 Exploratory data analysis

The ratings data is grouped by movie to calculate rating counts and average ratings. A minimum rating-count filter is applied when identifying highly rated movies so that movies with very few ratings are not treated as directly comparable to widely rated titles. Ratings are also grouped by user to identify active users, and the distribution of rating values is summarized and plotted.

6.3 Rating-matrix sparsity

Sparsity describes the proportion of possible user–movie rating pairs that are not observed. If U is the number of distinct users, M is the number of distinct movies, and R is the number of observed ratings, the sparsity percentage is calculated as:

$$\textbf{Sparsity (\%)} = [1 - R / (U \times M)] \times 100$$

6.4 ALS recommendation model

The ALS model uses three fields: User_Id, Movie_Id, and Rating. The ratings are split into training and test subsets using a fixed random seed for reproducibility. The baseline configuration used in the cleaned workflow is rank = 10, maxIter = 10, and regParam = 0.1. The model uses nonnegative factorization and coldStartStrategy='drop'.

ALS factorizes the user–item rating matrix into lower-dimensional user and item factor matrices. Their latent-factor interactions are used to estimate ratings and rank candidate movies for recommendation.

6.5 Model evaluation

The trained model transforms the held-out test set to produce predicted ratings. RegressionEvaluator calculates Root Mean Squared Error (RMSE), which summarizes the magnitude of prediction errors in rating units. A lower RMSE indicates better predictive accuracy when models are evaluated on the same data split.

6.6 Recommendation generation

The model's recommendForAllUsers(10) method generates up to ten recommendations per user. The nested recommendation results are expanded into individual rows and joined with movie metadata so that the selected user's recommendations can be displayed with movie titles, genres, and predicted scores.

7. System Workflow

1. SQLite database connection
2. Load source tables into Spark DataFrames
3. Data inspection and exploratory analysis
4. Prepare user, movie, and rating columns
5. Calculate rating-matrix sparsity and activity summaries
6. Split ratings into training and test sets
7. Train ALS collaborative-filtering model
8. Predict test ratings and calculate RMSE
9. Generate top-N recommendations and join movie metadata

8. Results and Discussion

The database loading stage was verified with the following record counts: 8,570 movies, 100,023 ratings, 8,570 links, and 2,488 tags. The exploratory workflow produces ranked tables for popular movies, average-rated movies, and active users, together with plots for movie popularity, user activity, and rating distribution.

The baseline ALS workflow is designed to train a model, produce test predictions, calculate RMSE, and generate user-specific recommendations. The exact RMSE value and example recommendation titles should be inserted after a complete successful run of the final notebook. They are intentionally not fabricated in this report.

Evaluation item	Value / status
Movies loaded	8,570
Ratings loaded	100,023
Links loaded	8,570
Tags loaded	2,488
Baseline ALS rank	10
Baseline ALS max iterations	10
Baseline ALS regularization	0.1
Test RMSE	To be filled after final notebook execution
Example user recommendations	To be filled after final notebook execution

9. Limitations

- The dataset is small and does not represent the scale or operational complexity of a commercial streaming platform.
- The baseline model uses explicit ratings and does not incorporate contextual information such as time, device, or session behavior.
- ALS performance depends on data coverage and hyperparameter choices.

- The cold-start strategy drops predictions for users or items that cannot be scored from the training model.
- The project runs Spark locally and does not demonstrate a multi-node cluster deployment.
- The final RMSE and recommendation examples must be recorded from the completed notebook run.

10. Future Enhancements

- Run systematic hyperparameter experiments and compare RMSE on a fixed test split.
- Add ranking metrics such as Precision@K and Recall@K.
- Improve genre analytics by splitting multi-genre strings into individual genres.
- Build a simple interface for selecting a user and displaying recommendations.
- Add visual comparisons of actual and predicted ratings.
- Evaluate the workflow on a larger dataset or a distributed Spark cluster.

11. Conclusion

The Movie Recommendation and User Behavior Analytics project demonstrates an end-to-end workflow that combines relational data loading, Spark-based exploratory analysis, collaborative filtering, model evaluation, and top-N recommendation generation. SQLite provides a convenient local data source, while PySpark and Spark MLlib provide the processing and modeling components. The implementation is suitable as an academic learning project and a foundation for further experimentation with recommendation algorithms and user behavior analytics.

References

- Apache Spark Documentation — https://spark.apache.org/docs/latest/
- Spark MLlib Collaborative Filtering — https://spark.apache.org/docs/latest/ml-collaborative-filtering.html
- MovieLens — https://grouplens.org/datasets/movielens/
- Python sqlite3 documentation — https://docs.python.org/3/library/sqlite3.html